Лабораторна робота №9

Семенюка Владислава Григоровича

Група ІПЗ-21

Варіант 7

Код консольної програми

```
using Lab9Models;

class Program
{
    static void Main()
    {
        const string divider =
"-----------------------------------------
-----------------------------------------";
        const int variant = 7;

        Console.WriteLine(divider);
        Console.WriteLine("Fullname: Vlad
Semeniuk");
        Console.WriteLine("Group: IPZ-21");
        Console.WriteLine("Variant: 7");
        Console.WriteLine(divider);

        SavingsAccount savingAccount = new
SavingsAccount(owner: $"Teacher-{variant}",
```

```
balance: 10000 + 500 * variant,
annualRatePercent: 5 + variant % 6);
        CreditAccount creditAccount = new
CreditAccount(owner: $"Customer-{variant}",
balance: 6000 + 300 * variant, annualFeePercent:
2 + variant % 5);

        Console.WriteLine("Hierarchy of the
classes");
        Console.WriteLine("ICalculable");
        Console.WriteLine("   ▲ realization");
        Console.WriteLine("BankAccount
[abstract]");

        Console.WriteLine("    |--
SavingsAccount");
        Console.WriteLine("       ▲ realization
of IReportable");
        Console.WriteLine("    |--
CreditAccount");
        Console.WriteLine("       ▲ realization
of IReportable");

        Console.WriteLine(divider);
```

```
        // Base Class Items
        Console.WriteLine("Iterate through the
base(abstract) class\n");
        BankAccount[] baseClassItems = {
savingAccount, creditAccount };
        foreach (BankAccount baseClassItem in
baseClassItems)
        {
            Console.WriteLine($"Item is:
{baseClassItem} =
{baseClassItem.GetDescription()}");
            Console.WriteLine($"Item is:
{baseClassItem} =
{baseClassItem.CalculateMetric()}");
        }
        Console.WriteLine(divider);


        // Interface Items
        Console.WriteLine("Iterate through the
base interface\n");
        ICalculable[] baseInterfaceItems =
        {
            savingAccount, creditAccount
        };
```

```
        foreach (ICalculable baseInterfaceItem in
baseInterfaceItems)
        {
            Console.WriteLine($"Item is:
{baseInterfaceItem} =
{baseInterfaceItem.CalculateMetric()}");
        }
        Console.WriteLine(divider);

        // Exp. If we set type for an object it
will be that type.
        // But we could use only methods or
properties which exist in that type
        // Factual type will always be the type
of the object which was initialized with a new
keyword

        // balance: 10000 + 500 * variant;
        // annualRatePercent: 5 + variant % 6;

        // Test for SavingsAccount
        // The formula for (CalculateMetric:
Balance * AnnualRatePercent / 100;)
```

```
        double testCalculateMetric = (10000 + 500
* variant) * (5 + variant % 6) / 100;
        Console.WriteLine($"Test for
(SavingsAccount-CalculateMatric) =
{testCalculateMetric}");
        Console.WriteLine("✅Correct!");
        Console.WriteLine(divider);

        // Individual assignment B
        IReportable[]  reportItems =
{savingAccount, creditAccount};
        foreach (IReportable reportItem in
reportItems)
        {

Console.WriteLine(reportItem.CreateReport());
        }
        Console.WriteLine(divider);

        // Exp. One class can use multiples
interfaces it can inherit only one
simple/abstract class
        // So in our case we use both interfaces.
We apply the  ICalculable for the base class
```

```
        // Then we add IReportable for our
inherited classes so they had this report
contract
        // We can assign those classes to
IReportable types and use only the contracted
methods and properties

    }
}
```

```
namespace Lab9Models;

public abstract class BankAccount : ICalculable
{
    public string Owner { get; }
    public double Balance { get; }

    protected BankAccount(
        string owner,
        double balance)
    {
```

```
        Owner = owner;
        Balance = balance;
    }


    public virtual string GetDescription()
    {
        return $"Description: owner: {Owner},
balance: {Balance}";
    }
    public abstract double CalculateMetric();
}
```

```
namespace Lab9Models;

public class CreditAccount : BankAccount,
IReportable
{
    public double AnnualFeePercent { get; }

    public CreditAccount(
        double annualFeePercent, string owner,
double balance)
        : base(owner, balance)
    {
        AnnualFeePercent = annualFeePercent;
    }
```

```
    public override string GetDescription()
    {
        return $"{base.GetDescription()}, annual:
{AnnualFeePercent}";
    }

    public override double CalculateMetric()
    {
        return this.Balance *
this.AnnualFeePercent / 100;
    }

    public string CreateReport()
    {
        return "Report---:   " +
this.GetDescription() + $" Calculate metric:
{this.CalculateMetric()}";
    }
}
```

```
namespace Lab9Models;


public interface ICalculable
{
```

```
    double CalculateMetric();
}
```

```
namespace Lab9Models;

public interface IReportable
{
    string CreateReport();
}
```

```
namespace Lab9Models;

public class SavingsAccount : BankAccount,
IReportable
{
    public double AnnualRatePercent { get; }

    public SavingsAccount(
        double annualRatePercent, string owner,
double balance)
        : base(owner, balance)
    {
        AnnualRatePercent = annualRatePercent;
    }


    public override string GetDescription()
```

```
    {
        return $"{base.GetDescription()}, annual:
{AnnualRatePercent}";
    }


    public override double CalculateMetric()
    {
        return Balance * AnnualRatePercent / 100;
    }

    public string CreateReport()
    {
        return "Report---:  " +
this.GetDescription() + $" Calculate metric:
{this.CalculateMetric()}";
    }


}
```

ШІ використовувався тільки для ресьорчу.